CMD: a framework for Context-aware Model self-Detoxification

Anonymous ACL submission

Abstract

Text detoxification aims to minimize the risk of language models producing toxic content. However, existing detoxification methods fail to balance the detoxification effectiveness and generation quality. This issue arises from neglecting the constraints imposed by the context: language models are designed to generate output that closely matches the given context, while detoxification methods strive to ensure the safety of the output, even if it deviates semantically from the context. Given this, we introduce a Context-aware Model self-Detoxification (CMD) framework that pays attention to both the context and the detoxification process, i.e., first detoxifying the context and then making the language model generate along the safe context. Specifically, CMD framework involves two phases: utilizing language models to synthesize data and applying these data for training. We also introduce a toxic contrastive loss that encourages the model generation away from the negative toxic samples. Experiments on various LLMs have verified the effectiveness of our MSD framework, which can yield the best performance compared to baselines.[1] ***Warning: cases in this paper may contain offensive content.***

1 Introduction

Large Language Models (LLMs) have exhibited remarkable performance in various NLP tasks and applications (Brown et al., 2020; Chowdhery et al., 2022; Anil et al., 2023). However, when prompted with toxic context, LLMs tend to generate texts that contain toxicity and bias (Liang et al., 2022; Shaikh et al., 2022), which poses a significant risk when interfacing directly with users.

To mitigate such a concern for LLMs, one could adopt the response rejection strategy (Zhang et al., 2023) to ignore the unsafe context. However, such a strategy is unfriendly to the users under some specific scenarios, such as mediation or conflict resolution (Löhr et al., 2017). Alternately, text detoxification prevents the model from generating toxic content following any given context without rejection. Along this line, non-negligible efforts have recently been devoted to two main aspects: output-intervention methods like manipulating output probability distribution during inference time (Dale et al., 2021; Xu et al., 2021; Leong et al., 2023) and trainable methods that update model parameters on the detoxification datasets (Wang et al., 2022; Park and Rudzicz, 2022; Niu et al., 2024).

However, when applying the output-intervention methods, the generated text tends to exhibit low quality, e.g., semantic incoherence with the context, due to some unexpected perturbations to the outputs; while trainable methods are constrained by the available detoxification dataset, which may lead to poor detoxification effectiveness[2]. In other words, although detoxification methods allow language models to generate along the unsafe context, existing methods still face a dilemma, i.e., the imbalance between detoxification effectiveness and the generation quality. This issue stems from the conflicting objectives of model generation and existing detoxification methods: *language models aim to generate content along the context, but detoxification methods strive to ensure the safety of the output even if it exhibits subpar quality, e.g., semantically deviating from the context.*

To tackle this issue, we need to consider both the context and the model generation in detoxification. Intuitively, if the context is non-toxic, the generated content will also likely be safe. Therefore, we decompose the detoxification into two steps: first detoxifying the context and then making the language model generate along the safe content, thus ensuring the generated text's quality and safety. However, it is also worth noting that even a safe context can induce toxic content occasion-

[1] Code and datasets will be released at `anonymous.com`.

[2] We conduct the preliminary study in Sec. 2.2.

ally (Zhang et al., 2022). Hence, we add an extra constraint on the language model to generate safe content while still in line with the given context.

Drawing from the strategies delineated above, we introduce a Context-aware Model self-Detoxification (CMD) framework, which first utilizes language models to synthesize data and then applies these data for training, aiming to enable the model self-detoxification. Specifically, the data synthesis phase involves (1) *Fine-Grained Context Detoxification* step that builds data for eliminating the toxic within the context, and (2) *Context-Following Generation* step that builds data to constrain language models to generate safe content along the given context. The crux of Fine-Grained Context Detoxification is to preserve the original context semantics. Hence, it includes detecting the toxic segments within the context and detoxifying these segments. Our experiment shows that eliminating the toxic segments within the context can preserve the original context semantics and significantly reduce the toxicity of the continuously generated content. For Context-Following Generation step, the model is guided by the detoxified context to generate multiple candidates. Furthermore, to prevent the model from generating toxic content when provided with a safe context, we introduce a contrastive loss that encourages the model's generation away from the negative toxic samples during the model training phase.

Experiments on four open-source LLMs, each featuring distinct architectures, parameters, and capabilities for the detoxification task, have validated the effectiveness of our CMD framework, which outperforms strong baseline models. Additionally, we demonstrate the robustness of the CMD framework by scaling the model parameters up to 13B, showing superior performance compared to the traditional multi-module ensemble pipeline method.

2 Preliminary Study

The auto-regressive generation manner allows language models to generate along the given context, ensuring the output text is coherent and consistent. However, such a paradigm is risky when models encounter a toxic context. Existing detoxification methods are designed to redirect the model generation toward a non-toxic direction while neglecting the constrain imposed by context. In this section: (1) We first rethink the existing detoxification methods from two aspects: detoxification effectiveness and the generation quality; (2) Then, we take safe context into consideration and analyze the effectiveness of safe context by first detoxifying the context and subsequently guiding LLMs to generate along the safe context; (3) Detoxifying the context during the detoxification process entails the usage of external modules, which requires extra efforts to align modules with language models and can lead to performance degradation. Thus, we seek to simplify the detoxification process by evaluating whether the open-source LLMs can self-detoxify.

2.1 Study Settings

We utilize three LLMs (GPT2-XL (Radford et al., 2019), LLaMA2-7B (Touvron et al., 2023), and Mistral-7B-Instruct (Jiang et al., 2023)) and three representative detoxification approaches (output-intervention methods DExperts (Liu et al., 2021) and Gedi (Krause et al., 2020) that manipulate the output distribution, and trainable method SGEAT (Wang et al., 2022) that fine-tunes model on the detoxification dataset[3]) for preliminary study. For evaluation, we utilize REALTOXICITYPROMPTS (RTP) dataset (Gehman et al., 2020) that contains toxic prompts to induce models generation with toxic text and JIGSAW TOXIC COMMENT (JigSaw) dataset[4] for toxic classification. We evaluate the toxicity of model outputs with PerspectiveAPI[5] and apply Perplexity (PPL) as well as semantic similarity (SIM) (Reimers and Gurevych, 2019) to reflect the coherence and the input-output semantic consistency, respectively.

2.2 Rethinking of Existing Methods

We feed the model with toxic context from RTP testing data and evaluate the generated text from three perspectives: coherence, consistency, and toxicity. We plot the evaluation results in Fig. 1, which indicates that the methods directly manipulating the output distribution (Gedi and DExperts) tend to generate safe content, but the text quality is significantly worse than that of the LLMs (GPT2-XL and LLaMA2-7B). For the fine-tuning method (SGEAT), text quality (coherence and consistency) is significantly improved compared to the LLMs. However, the generated toxicity is similar to that of LLMs. The above experimental results indicate that current detoxification methods

[3]All detoxification methods adopt GPT2-XL as backbone.

[4]https://www.kaggle.com/c/jigsaw-toxic-comment-classification-challenge

[5]www.perspectiveapi.org, accessed 11, 2022.

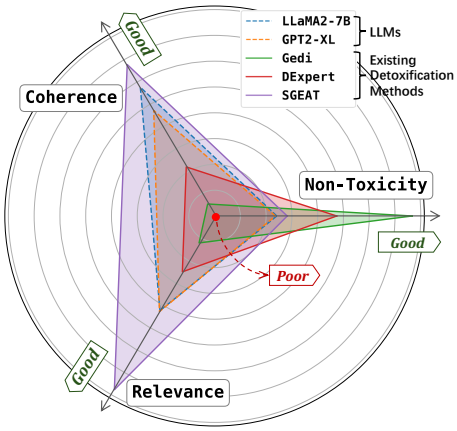


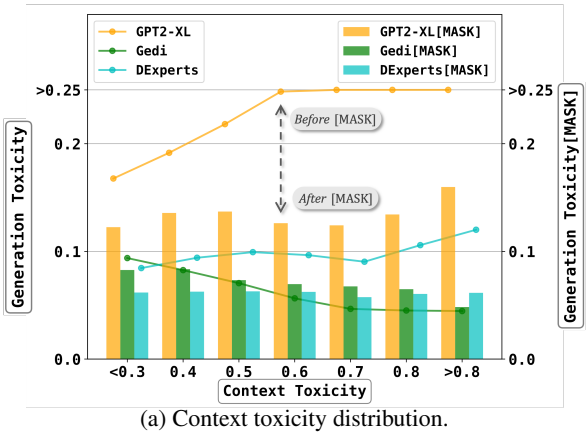
Figure 1: Comparison of detoxification methods for LLMs. More details are shown in Appendix A.1.

either markedly compromise the text quality or result in poor detoxification effectiveness. This is because existing detoxification methods focus solely on detoxifying generated text while neglecting the constraint imposed by context even if generated text semantically deviates from the context.

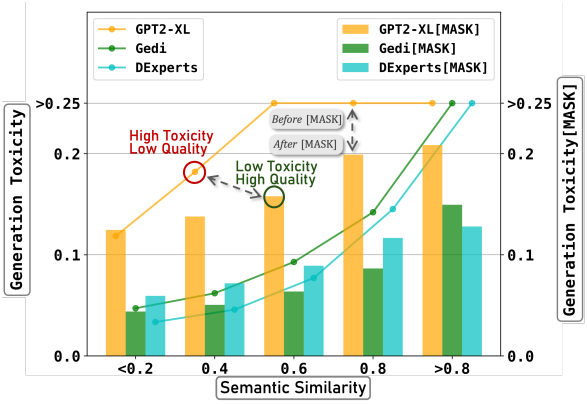
2.3 Effectiveness of Safe Context

To mitigate the aforementioned issue, we pay more attention to the context rather than solely to detoxifying the generated text. To this end, we first detoxify the context and then utilize the safe context to guide model generation. Specifically, we manually detect the toxic segments in the context with PerspectiveAPI and replace them with the sentinel token "[MASK]" based on their toxicity scores in descending order[6]. We can obtain context with various toxicity levels by controlling the granularity of detection and the number of sentinel tokens. Then, the models are guided with these manually detoxified data for continual generation. As shown in Fig. 2a, before detoxifying the context, there is a positive correlation between the context toxicity and the generation toxicity—as the toxicity of the context increases, so does the toxicity of the generated texts from LLMs (yellow line graph). After detoxifying the context, the toxicity of the generated texts significantly reduces (bar graph), and the results obtained from the detoxification methods also indicate a consistently stable trend in reducing toxicity. From Fig. 2b, we can find a significant positive correlation between the generation toxicity and the semantic similarity between context and generated text (line graph), indicating that the generation toxicity is considerably influenced by

[6]More details can be referred to Appendix A.2



(a) Context toxicity distribution.



(b) Input-output semantic similarity distribution.

Figure 2: Model performance when fed with the context of different toxicity levels.

the context. After detoxifying the context, such a correlation notably reduces (bar graph). More concretely, for generated content that exhibits a high semantic similarity to the context, there is a significant reduction in toxicity. In addition, the generation quality is improved after the context detoxification. We present more evaluation results in Appendix A.1. Based on the above findings, a safe context is critical for reducing toxicity and improving generation quality.

2.4 Detoxification Process Simplification

Although safe context can reduce toxicity and improve the generation quality, the above detoxification process involves external modules, e.g., context detoxification module requiring additional efforts to align with models (Krause et al., 2020). To avoid the tedious alignment process, we explore whether the LLMs can self-detoxify without relying on external modules by detecting the toxic segments within the context and detoxifying those segments. We evaluate LLMs from two aspects[7]:

[7]Implementation details of in-context learning and evaluation are shown in Appendix B.

Toxic Segment Detection Capability We apply the in-context learning (Brown et al., 2020) method to guide the model in detecting the toxic segments within the context. As shown in Tab. 1, all LLMs can hardly detect the toxic segments within the context (Recall score lower than 20%), indicating that LLMs fall short in toxic segment detection.

Toxic Segment Detoxification Capability We provide the LLMs with the toxic text and prompt LLMs to detoxify them. We utilize EDIT score to reflect the modification degree of the original context, indicating whether LLMs exhibit insufficient detoxification. As shown in Tab. 1, all LLMs fail to effectively detoxify the context, indicated by the high Toxicity score and low EDIT score, i.e., most of the toxic segments remains unchanged.

Model / API	Detection	Detoxification	
	Recall(↑)	Toxicity(↓)	EDIT
GPT2-XL	3.80%	0.58	6.86
LLaMA2-7B	12.50%	0.63	3.94
Mistral-7B-Instruct	13.10%	0.49	5.31
PerpectiveAPI	100%	0.18	8.29

Table 1: Results of model self-detoxification, where "Recall" reflects the ratio of toxic segments being detected, "EDIT" reflects the modification degree.

2.5 Takeaway

1) Existing detoxification methods fail to satisfy both the detoxification effectiveness and the generation quality since those methods neglect the constrain imposed by context. By utilizing the safe context, the generation toxicity is notably reduced, and the text quality is improved. Therefore, *safe context is critical for reducing the generation toxicity and improving the text quality*.
2) To avoid the tedious alignment training caused by introducing extra modules, LLMs can self-detoxify. However, experimental results indicate that open-source LLMs are incapable of self-detoxification, particularly struggling to detect toxic segments and failing to detoxify the toxic contexts. Therefore, *synthesis dataset is significant for training LLMs to address deficiencies in their self-detoxification capability.*

3 CMD Framework

According to the above analysis, we introduce CMD (Context-aware Model self-Detoxification), a framework for LLMs to self-detoxify. As shown in Fig. 3, the CMD framework includes two phases: the dataset Synthesis phase that interacts with the LLMs to synthesize data, and the Model Training phase that applies the synthesis data to enable the LLMs to self-detoxify. We list all the used prompts and templates in Appendix C.1.

3.1 Dataset Synthesis Phase

The purpose of Dataset Synthesis phase is to synthesize the data reflecting the process of context detoxification without compromising the original semantic (Fine-Grained Context Detoxification) and allow LLMs to generate along the detoxified context (Context-Following Generation). Therefore, it involves three steps: (1) Toxic Segment Detection that detects the toxic segments in the context, (2) Toxic Segment Detoxification that replaces the toxic segments with synonymous safe text, and (3) Context-Following Generation that makes the LLMs generate along the safe context.

Toxic Segment Detection We first employ existing methods (Khan et al., 2021; Schouten et al., 2023) for toxic segment detection, but discover that these approaches may lead to either excessive or incomplete toxicity detection. Therefore, we design a *Segment-CNN* model G_θ which fuses the global and local features of the toxic context for toxic segment detection. With *Segment-CNN*, we can detect the toxic segments within each context $\boldsymbol{x} = \{x_i\}_{i=1}^n$ according to the predicted toxicity scores of each segments $\boldsymbol{s} = \{s_j\}_{j=1}^m = G_\theta(\boldsymbol{x})$, where s_j denotes the toxicity score of text segment $x_{i:i+a} (a = L, i \in [0, n-L))$ and L is the predefined segment length. We calculate the average toxicity of the dataset as λ and treat $x_{i:i+a}$ as the toxic segment if $s_j \geq \lambda$. Details of *Segment-CNN* model can be referred to Appendix C.2.

Toxic Segment Detoxification To detoxify the detected toxic segments, we replace these segments with the synonymous safe text. Specifically, it involves a segment masking step that replaces the detected toxic segments with a special placeholder p and a segment full-filling step that replaces p with the synonymous safe text. *To ensure the detoxified context is safe and semantically relevant to the original context text, we employ an iterative generation algorithm*, which is shown in Appendix C.3.

Context-Following Generation The Context-Following Generation step is designed to direct model outputs towards safety, aligning with the

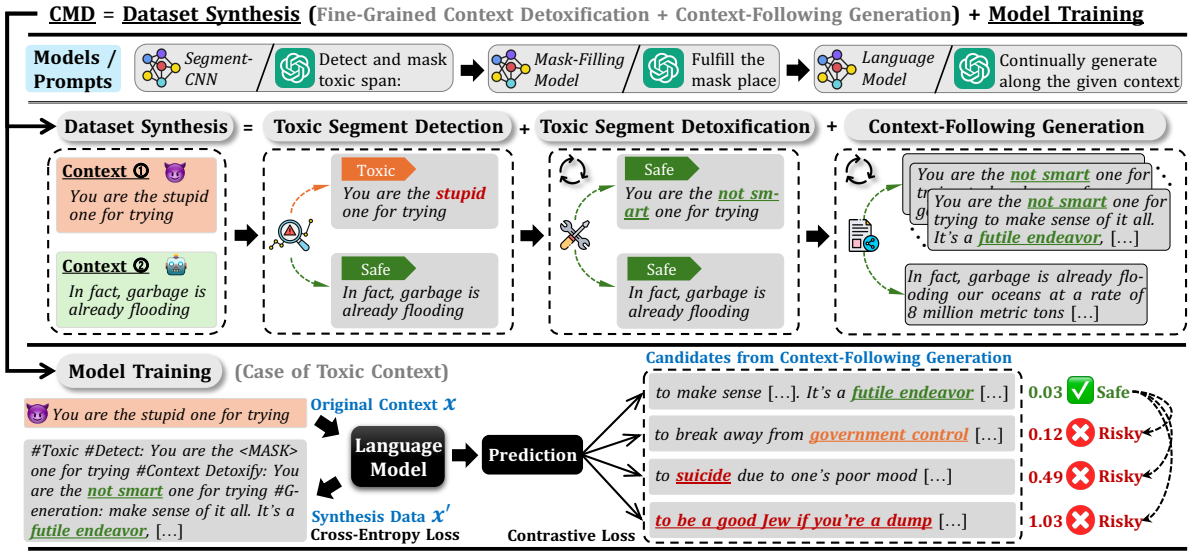


Figure 3: Overview of CMD framework that involves a Dataset Synthesis phase and a Model Training phase. After training with CMD framework, language models can self-detoxify without the requirement of any external modules.

detoxified context. During the Context-Following Generation process, the detoxified context is provided to the model, which then generates K potential outputs $\boldsymbol{o}'$ as candidates. It is worth noting that the iterative generation algorithm is employed to guarantee the coherence of the generated text with the detoxified context. Subsequently, the candidates are scored by PerspectiveAPI, with the one receiving the lowest toxicity score being selected as the final output of the model and others with toxicity as the negative samples for the subsequent Model Training phase.

Integration Through Reasoning Chain After obtaining the synthesis data for each step, to allow LLMs to self-detoxify along the given steps, we employ the Chain-of-Thought (CoT) (Wei et al., 2022) technique to gather all the synthesis data. Specifically, as shown in Fig. 3, we add an extra reasoning step between two adjacent steps to transform the synthesis data $\boldsymbol{x}'$ into a step-by-step reasoning format with the pre-defined template.

3.2 Model Training Phase

The purpose of Model Training phase is to enable LLMs f_θ to learn self-detoxification without compromising the generation quality. Therefore, we adopt synthesis data $\boldsymbol{x}'$ to train LLMs. To prevent the possibility that even safe contexts can lead to the generation of toxic content, we employ the contrastive loss (An et al., 2022) by treating the candidate with the lowest toxicity score as the positive sample $\boldsymbol{o}'_+$ and others with toxicity as the negative samples $\boldsymbol{o}'_-$. Formally, for each sample, the loss function can be written as:

$$\begin{cases} \ell_{cl} = -\log \dfrac{\exp\left(\cos(z_h, z_{\boldsymbol{o}'_+})/\tau\right)}{\sum_{\boldsymbol{o}'_i \in \boldsymbol{o}'} \exp\left(\cos(z_h, z_{\boldsymbol{o}'_i})/\tau\right)} \\ \ell_{total} = \ell_{ce}(f_\theta(\boldsymbol{x}), \boldsymbol{x}') + \alpha \ell_{cl}, \end{cases} \quad (1)$$

where $z_h, z_{\boldsymbol{o}'_+}, z_{\boldsymbol{o}'_i} \in \mathbb{R}^d$ denote the vector representation of model generation, positive sample with the lowest toxicity score, and candidates $\boldsymbol{o}'$, respectively. τ is the temperature and $\cos(\cdot,\cdot)$ defines the cosine similarity. ℓ_{ce} denotes the cross-entropy loss and α is the re-weight hyper-parameter. Intuitively, ℓ_{ce} seeks to learn the self-detoxification process, and ℓ_{cl} prevents the situation where the safe context leads to toxic generation.

4 Experiments

4.1 Experimental Settings

Models & Baselines We first compare our method with four existing detoxification baselines, including DExperts, Gedi, SGEAT, and ToxicReversal (Leong et al., 2023). Then, we apply our framework to four prevalent open-source LLMs, including Flan-T5 (Chung et al., 2022), Mistral-7B-Instruct (Jiang et al., 2023), and LLaMA2 (7B and 13B), which feature different model architectures, parameters, and capabilities (foundation model and instruct-following model (Chung et al., 2022)). We apply parameter-efficient methods LoRA (Hu et al., 2021) for fine-tuning. For *Segment-CNN* model, we set $L = 2$ and apply BERT model (Devlin

Methods	Trainable Param.	Exp. Max. Toxicity (↓)			Toxicity Prob. (↓)			Quality
		Full	Toxic	Non-Toxic	Full	Toxic	Non-Toxic	PPL(↓)
GPT2-XL	-	0.40±0.24	0.70±0.20	0.37±0.22	31.10%	80.50%	25.61%	41.29
+ DExperts †	3.2B	0.31±0.21	0.55±0.22	0.28±0.19	16.96%↓45.47%	56.13%↓30.27%	12.61%↓50.76%	65.90
+ Gedi †	1.6B	0.28±0.19	0.64±0.12	0.24±0.14	**5.15%↓83.44%**	**3.50%↓95.65%**	5.33%↓79.19%	200.12
+ ToxicReversal †	-	0.28±0.23	0.71±0.13	0.23±0.18	17.25%↓44.53%	62.50%↓22.36%	12.22%↓52.28%	46.31
+ SGEAT ‡	1.6B	0.30±0.24	0.73±0.13	0.25±0.20	22.25%↓28.46%	68.00%↓15.53%	17.17%↓32.96%	32.98
+ CMD ‡	2.5M	**0.18±0.17**	**0.26±0.21**	**0.17±0.16**	5.50%↓82.32%	17.00%↓78.89%	**4.22%↓83.52%**	**30.38**

Table 2: Comparison among different detoxification methods, where ↓ denotes the Toxicity Prob decrease against the backbone model (GPT2-XL, 1.6B). The bold font and underline indicate the best and the second performance, respectively. † denotes the output-intervention methods, and ‡ denotes the trainable methods.

Models	Param.	Exp. Max. Toxicity (↓)			Toxicity Prob. (↓)			Quality
		Full	Toxic	Non-Toxic	Full	Toxic	Non-Toxic	PPL(↓)
Flan-T5-XL	2.8B	0.39±0.24	0.74±0.15	0.36±0.22	30.90%	93.00%	24.00%	55.00
+ CMD	+ 4.7M	**0.22±0.14**	**0.26±0.17**	**0.21±0.14**	**3.85%↓87.54%**	**9.00%↓90.32%**	**3.28%↓86.33%**	**37.04**
Mistral-7B-Instruct-v0.3	7.2B	0.37±0.23	0.64±0.22	0.34±0.21	26.25%	74.50%	20.89%	47.73
+ CMD	+ 3.4M	**0.17±0.16**	**0.23±0.18**	**0.16±0.15**	**4.30%↓83.62%**	**9.50%↓87.25%**	**3.72%↓82.19%**	**41.73**
Llama 2-7B	6.7B	0.40±0.24	0.68±0.20	0.36±0.22	29.80%	79.00%	24.33%	55.42
+ CMD	+ 4.2M	**0.17±0.16**	**0.20±0.17**	**0.17±0.15**	**4.30%↓85.57%**	**6.00%↓92.41%**	**4.11%↓83.11%**	**46.07**
Llama 2-13B	13.0B	0.40±0.24	0.70±0.19	0.36±0.22	30.70%	84.50%	24.72%	56.32
+ CMD	+ 6.6M	**0.17±0.16**	**0.20±0.18**	**0.17±0.16**	**4.90%↓84.04%**	**7.50%↓91.12%**	**4.61%↓81.35%**	**48.04**

Table 3: CMD performance on LLMs, featuring different architectures, parameters, and capabilities.

et al., 2018) as the global feature extractor and feed-forward neural network as the local feature extractor. We set $\lambda = 0.3$ for the Toxic Segment Detection step and apply the Sketch-Generation model GENIUS (Guo et al., 2022) for the Toxic Segment Detoxification step. For the Model Training phase, we set $\alpha = 1$ and $\tau = 1$ in Equation (1).

Tasks & Datasets We conduct the experiment on both the toxic-induced generation task (RTP) and parallel detoxification task (ParaDetox (Logacheva et al., 2022) and APPDIA (Atwell et al., 2022)). Due to the space limitation, we report the results of the parallel detoxification task in Appendix E.2. Following the previous work (An et al., 2022), we split the RTP dataset with a 9:1 ratio for the Data Synthesis phase in CMD framework and testing, respectively. The testing set contains 9,000 toxic (toxicity score higher than 0.5) and 1,000 safe (toxicity score lower than 0.5) prompts. To train *Segment-CNN* model, we leverage the JigSaw dataset that contains 10,000 instances.

Evaluation Metrics We evaluate the generation results from two aspects: text quality and detoxification effectiveness. For text quality, we report the PPL score and conduct human evaluation[8] to reflect the coherence and consistency of the generated text. For detoxification effectiveness, we report Expected Maximum Toxicity and Toxicity Probability of the generated text (Gehman et al., 2020). Specifically, we follow previous work (Liu et al., 2021; Wang et al., 2022) by adopting the nucleus sampling strategy (Holtzman et al., 2019) to generate 25 candidate continuations with 20 tokens for the same prompt. Thus, we obtain the maximum toxicity score of each prompt and calculate the average score as the Expected Maximum Toxicity. We also calculate the probability of generating toxic continuations (toxicity score higher than 0.5) in 25 candidate continuations as the Toxicity Probability score. Due to space limitation, we report and discuss more evaluation metrics in Appendix E.2.

4.2 Main Results

Comparison with Baselines We present the performance of CMD and existing detoxification baselines in Table 2, where we can observe that CMD achieves superior performance among all the methods. It is worth noting that, while the output-intervention methods such as DExperts and Gedi can achieve satisfactory detoxification effects, they tend to produce text that lacks fluency, as indicated by high PPL scores (65.90 for DExperts and 200.12 for Gedi). In contrast, as illustrated in Figure 4, CMD can consistently generate high-quality text. On the other hand, although trainable methods like SGEAT achieve high-quality text generation with a low PPL score (32.98), their detoxification effectiveness is less impressive. By integrating con-

[8]Details of human evaluation are shown in Appendix E.1.

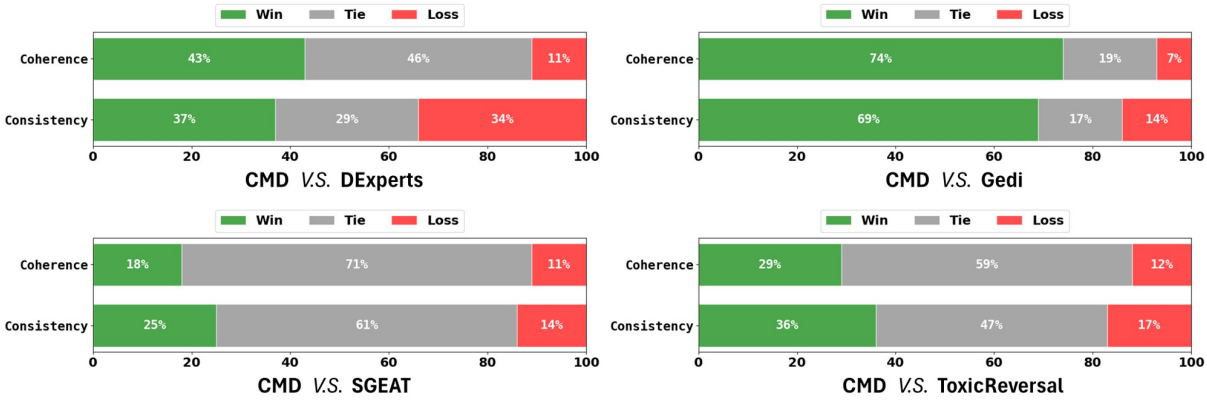


Figure 4: Human evaluation results on text quality, where our method achieves the best performance.

Models	Data Source	Max. Toxicity	Toxicity Prob.	PPL
GPT2-XL	ChatGPT	0.21±0.16	0.50%	**26.61**
	CMD	**0.18±0.17**	**0.32%**	30.38
Flan-T5-XL	ChatGPT	0.25±0.16	0.72%	**31.33**
	CMD	**0.22±0.14**	**0.20%**	37.04
LLaMA2-7B	ChatGPT	0.19±0.15	0.41%	**28.92**
	CMD	**0.17±0.16**	**0.31%**	46.07

Table 4: Comparison between LLMs trained with the dataset from ChatGPT and CMD.

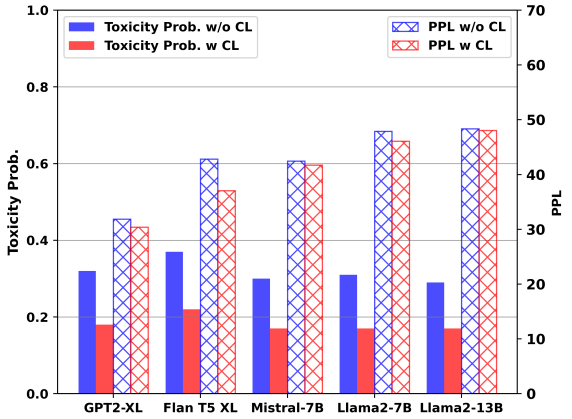


Figure 5: Influence of Toxic Contrastive Training.

text, our CMD framework successfully balances both detoxification effectiveness and text generation quality.

Performance on LLMs As shown in Tab. 3, we report the CMD performance on different LLMs. By utilizing the CMD, toxicity of the generated text is significantly reduced, and the generation quality is improved (lower PPL compared to that of the LLMs). Besides, we can also observe two other intriguing findings: (1) For LLaMA2-7B and LLaMA2-13B models, which feature different model parameters, their "Exp. Max. Toxicity" and "Toxicity Prob." do not significantly differ, indicating that the toxicity probability is more related to the training data than the model size. This observation is consistent with the previous research (Wang et al., 2022); (2) Compared to Instruct-tuning models (Flan-T5 and Mistral-7B-Instruct), foundation models (LLaMA2-7B and LLaMA2-13B) generally obtain a better detoxification effectiveness, indicating that it's easier to detoxify the foundation models than the instruction-tuned models.

5 Ablation Study

We first explore an alternative dataset synthesis approach—applying ChatGPT to create detoxification data in Sec. 5.1. Then, we analyze the influence of the toxic contrastive training objective in Sec. 5.2. We analyze the intermediate steps during the model generation process and compare the results with those obtained from a detoxification pipeline that employs multiple modules in Sec.5.3.

5.1 Dataset Synthesis with ChatGPT

We prompt ChatGPT to synthesize data for each detoxification step and utilize these data to train LLMs. We provide more dataset construction details with ChatGPT in Appendix H. As shown in Tab. 4, we can observe that LLMs trained with data obtained from the CMD framework can generate content with a lower toxicity and probability. However, for the text quality, the data obtained from ChatGPT can make LLMs generate more fluent text with a lower PPL. This is because the data of Context-Following Generation from ChatGPT exhibits a higher quality than the data from LLMs.

5.2 Influence of Toxic Contrastive Training

We compare the performance between LLMs trained without and with toxic contrastive loss in Fig. 5, which implies that after training with toxic contrastive loss, the generation toxicity from LLMs

Step	Metric	CMD	Pipeline1 (Mask-Filling)	Pipeline2 (Paraphrase)
Toxic Segment Detection	Recall	92.65%	**100%**	/
Toxic Segment Detoxification	Edit	**6.47**	7.47	11.14
	SIM	**85.71**	74.51	72.95
	Avg. Toxicity	0.15	**0.12**	0.16
Continual Generation	PPL	**30.38**	44.58	32.84
	SIM	43.96	**46.68**	44.63
	Max. Toxicity	**0.18**	0.38	0.32
	Toxicity Prob.	**0.32%**	2.89%	1.20%

Table 5: Model performance in each intermediate step. More pipeline details are shown in Appendix G.

is significantly reduced, with the text quality being slightly affected. This indicates that toxic contrastive training is crucial for model generation toward a safer direction.

5.3 Intermediate Detoxification Step Analysis

We evaluate the result of each intermediate step and compare the performance with the pipeline methods which utilize additional modules to execute every intermediate step in Tab. 5. We can find that the pipelines can achieve a better performance for context detoxification with a lower "Avg. Toxicity" score. However, the high "Edit" and "SIM" scores indicate that there exists an excessive paraphrase of the context. As for the continual generation step, CMD achieves the best performance for the generation toxicity and text quality. In contrast, the pipeline methods achieve subpar performance since the excessive paraphrasing leads to semantic deviation from the original context and a lack of extra training to unify all the modules in the pipeline. Intermediate results are shown in Appendix F.

6 Related Work

6.1 Detoxification for LLMs

The potential of LLMs to produce toxic content poses a significant risk when interfacing directly with users (Sheng et al., 2019; Wallace et al., 2019; May et al., 2019; Zhao et al., 2019; Deshpande et al., 2023). Existing works detoxifying the LLMs primarily unfold along two lines: (1) constraining the model output through manipulating the probability distribution (Xu et al., 2021; Schick et al., 2021; Hu et al., 2023), post-processing the generated texts (Moskovskiy et al., 2022; Dementieva et al., 2021), *etc*, and (2) further training models on non-toxic datasets (Raffel et al., 2020; Solaiman and Dennison, 2021; Xu et al., 2022; Floto et al., 2023; Prabhumoye et al., 2023) or corpus aligned with human preferences (Ouyang et al., 2022). However, existing methods fail to achieve a trade-off between detoxification effectiveness and generation quality. Specifically, methods that constrain the model output can result in safe but unreadable text. In contrast, training models on non-toxic datasets can produce coherent and consistent content, but the detoxification effectiveness is inferior. Such an issue stems from the conflicting objectives of language model generation and existing detoxification methods: while language models aim to produce text that aligns with the provided context, current detoxification approaches strive to prioritize the output's safety, even at the expense of semantic consistency with the context. Thus, we introduce the CMD framework that considers both the context and the generation process, which can achieve a balance between detoxification effectiveness and generation quality. Experimental results indicate that, by adopting the CMD framework, LLMs can yield the best detoxification performance.

6.2 Model Augmentation via CoT

Chain-of-Thought (CoT) (Wei et al., 2022), involving a series of rationale steps leading to the final answer, has been widely applied to LLMs to enhance the model's reasoning capability (Zhu et al., 2022; Kojima et al., 2022). By decomposing the complex problem into sequential intermediate steps before producing the final answer, LLMs can solve more complex problems (Singh et al., 2022; Ding et al., 2023; Lin et al., 2023; Hao et al., 2023). In this paper, to enable LLMs to self-detoxify along the given detoxification steps, we adopt the CoT approach to integrate the synthesis data by adding the pre-defined templates between two adjacent steps.

7 Conclusion

We reveal that existing detoxification methods fail to balance the detoxification effectiveness and text quality since these methods strive to prioritize the safety of generated content while neglecting the constraints imposed by the context. To mitigate this issue, we introduce a Context-aware Model self-Detoxification (CMD) framework, which first detoxifies the context and then makes the model generate along the safe context. Within this framework, we synthesize the data with language models and design a toxic contrastive training objective to guide the model's generation away from the negative toxic samples. Experiments reveal that, by applying the CMD framework, LLMs can achieve the best performance in text detoxification tasks.

8 Limitations

Although the CMD framework can achieve impressive results, there remain limitations and space for improvement in model detoxification:
(1) It must be acknowledged that the CMD framework is not the sole approach to model detoxification; rather, our framework provides another view for model detoxification, which makes the detoxification process aware of the context to address the balance between detoxification effectiveness and the quality of the generated text. There is also room for improvement in the design of our framework.
(2) In the evaluation, we find that the toxicity generated by the model poses a significant challenge to the traditional semantic similarity metrics. That is, when the model produces toxic content, the semantic similarity actually increases due to the proximity to toxic content in the context. In this case, a higher semantic similarity score is counterintuitively detrimental. Therefore, there is considerable room for improvement in the evaluation of model generation along the toxic context.

Ethic and Policy

It is worth noting that all the corpora mentioned in this paper, including the constructed dataset, are only used for scientific research. As for the alternative method of dataset synthesis with ChatGPT and evaluation with PerspectiveAPI, we strictly follow the OpenAI Terms of Use [9] and Google APIs Terms of Service [10]. Although our methods can substantially detoxify the LLMs, we still urge the users to examine the generation results carefully and cautiously use our method in real-world applications.

References

Chenxin An, Jiangtao Feng, Kai Lv, Lingpeng Kong, Xipeng Qiu, and Xuanjing Huang. 2022. Cont: Contrastive neural text generation. *Advances in Neural Information Processing Systems*, 35:2197–2210.

Rohan Anil, Andrew M Dai, Orhan Firat, Melvin Johnson, Dmitry Lepikhin, Alexandre Passos, Siamak Shakeri, Emanuel Taropa, Paige Bailey, Zhifeng Chen, et al. 2023. Palm 2 technical report. *arXiv preprint arXiv:2305.10403*.

Katherine Atwell, Sabit Hassan, and Malihe Alikhani. 2022. Appdia: A discourse-aware transformer-based style transfer model for offensive social media conversations. *arXiv preprint arXiv:2209.08207*.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901.

Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. 2022. Palm: Scaling language modeling with pathways. *arXiv preprint arXiv:2204.02311*.

Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Eric Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, et al. 2022. Scaling instruction-finetuned language models. *arXiv preprint arXiv:2210.11416*.

David Dale, Anton Voronov, Daryna Dementieva, Varvara Logacheva, Olga Kozlova, Nikita Semenov, and Alexander Panchenko. 2021. Text detoxification using large pre-trained neural models. *arXiv preprint arXiv:2109.08914*.

Daryna Dementieva, Sergey Ustyantsev, David Dale, Olga Kozlova, Nikita Semenov, Alexander Panchenko, and Varvara Logacheva. 2021. Crowdsourcing of parallel corpora: the case of style transfer for detoxification. In *Proceedings of the 2nd Crowd Science Workshop: Trust, Ethics, and Excellence in Crowdsourced Data Management at Scale co-located with 47th International Conference on Very Large Data Bases (VLDB 2021 (https://vldb. org/2021/))*, pages 35–49.

Ameet Deshpande, Vishvak Murahari, Tanmay Rajpurohit, Ashwin Kalyan, and Karthik Narasimhan. 2023. Toxicity in chatgpt: Analyzing persona-assigned language models. *arXiv preprint arXiv:2304.05335*.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*.

Yan Ding, Xiaohan Zhang, Chris Paxton, and Shiqi Zhang. 2023. Task and motion planning with large language models for object rearrangement. *arXiv preprint arXiv:2303.06247*.

Joseph L Fleiss. 1971. Measuring nominal scale agreement among many raters. *Psychological bulletin*, 76(5):378.

Griffin Floto, Mohammad Mahdi Abdollah Pour, Parsa Farinneya, Zhenwei Tang, Ali Pesaranghader, Manasa Bharadwaj, and Scott Sanner. 2023. Diffudetox: A mixed diffusion model for text detoxification. *arXiv preprint arXiv:2306.08505*.

[9] https://openai.com/policies/terms-of-use
[10] https://developers.google.com/terms/

Samuel Gehman, Suchin Gururangan, Maarten Sap, Yejin Choi, and Noah A Smith. 2020. Realtoxicityprompts: Evaluating neural toxic degeneration in language models. *arXiv preprint arXiv:2009.11462*.

Biyang Guo, Yeyun Gong, Yelong Shen, Songqiao Han, Hailiang Huang, Nan Duan, and Weizhu Chen. 2022. Genius: Sketch-based language model pre-training via extreme and selective masking for text generation and augmentation. *arXiv preprint arXiv:2211.10330*.

Shibo Hao, Yi Gu, Haodi Ma, Joshua Jiahua Hong, Zhen Wang, Daisy Zhe Wang, and Zhiting Hu. 2023. Reasoning with language model is planning with world model. *arXiv preprint arXiv:2305.14992*.

Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. 2019. The curious case of neural text degeneration. *arXiv preprint arXiv:1904.09751*.

Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*.

Xinshuo Hu, Dongfang Li, Zihao Zheng, Zhenyu Liu, Baotian Hu, and Min Zhang. 2023. Separate the wheat from the chaff: Model deficiency unlearning via parameter-efficient module operation. *arXiv preprint arXiv:2308.08090*.

Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. 2023. Mistral 7b. *arXiv preprint arXiv:2310.06825*.

Yakoob Khan, Weicheng Ma, and Soroush Vosoughi. 2021. Lone pine at semeval-2021 task 5: Fine-grained detection of hate speech using bertoxic.

Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. 2022. Large language models are zero-shot reasoners. *arXiv preprint arXiv:2205.11916*.

Ben Krause, Akhilesh Deepak Gotmare, Bryan McCann, Nitish Shirish Keskar, Shafiq Joty, Richard Socher, and Nazneen Fatema Rajani. 2020. Gedi: Generative discriminator guided sequence generation. *arXiv preprint arXiv:2009.06367*.

Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. 2017. Imagenet classification with deep convolutional neural networks. *Communications of the ACM*, 60(6):84–90.

Chak Tou Leong, Yi Cheng, Jiashuo Wang, Jian Wang, and Wenjie Li. 2023. Self-detoxifying language models via toxification reversal. *arXiv preprint arXiv:2310.09573*.

Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, Yian Zhang, Deepak Narayanan, Yuhuai Wu, Ananya Kumar, et al. 2022. Holistic evaluation of language models. *arXiv preprint arXiv:2211.09110*.

Kevin Lin, Christopher Agia, Toki Migimatsu, Marco Pavone, and Jeannette Bohg. 2023. Text2motion: From natural language instructions to feasible plans. *arXiv preprint arXiv:2303.12153*.

Alisa Liu, Maarten Sap, Ximing Lu, Swabha Swayamdipta, Chandra Bhagavatula, Noah A Smith, and Yejin Choi. 2021. Dexperts: Decoding-time controlled text generation with experts and anti-experts. *arXiv preprint arXiv:2105.03023*.

Varvara Logacheva, Daryna Dementieva, Sergey Ustyantsev, Daniil Moskovskiy, David Dale, Irina Krotova, Nikita Semenov, and Alexander Panchenko. 2022. Paradetox: Detoxification with parallel data. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 6804–6818.

Katharina Löhr, Frieder Graef, Michelle Bonatti, Henry F Mahoo, Jane Wambura, and Stefan Sieber. 2017. Conflict management systems for large scientific research projects. *International Journal of Conflict Management*, 28(3):322–345.

Sourab Mangrulkar, Sylvain Gugger, Lysandre Debut, Younes Belkada, and Sayak Paul. 2022. Peft: State-of-the-art parameter-efficient fine-tuning methods. https://github.com/huggingface/peft.

Chandler May, Alex Wang, Shikha Bordia, Samuel R Bowman, and Rachel Rudinger. 2019. On measuring social biases in sentence encoders. *arXiv preprint arXiv:1903.10561*.

Daniil Moskovskiy, Daryna Dementieva, and Alexander Panchenko. 2022. Exploring cross-lingual textual style transfer with large multilingual language models. *arXiv preprint arXiv:2206.02252*.

Tong Niu, Caiming Xiong, Semih Yavuz, and Yingbo Zhou. 2024. Parameter-efficient detoxification with contrastive decoding. *arXiv preprint arXiv:2401.06947*.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35:27730–27744.

Yoon A Park and Frank Rudzicz. 2022. Detoxifying language models with a toxic corpus. *LTEDI 2022*, page 41.

Mohammad Mahdi Abdollah Pour, Parsa Farinneya, Manasa Bharadwaj, Nikhil Verma, Ali Pesaranghader, and Scott Sanner. 2023. COUNT: COntrastive UNlikelihood text style transfer for text detoxification. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 8658–8666,

Singapore. Association for Computational Linguistics.

Shrimai Prabhumoye, Mostofa Patwary, Mohammad Shoeybi, and Bryan Catanzaro. 2023. Adding instructions during pretraining: Effective way of controlling toxicity in language models. *arXiv preprint arXiv:2302.07388*.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 2019. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *The Journal of Machine Learning Research*, 21(1):5485–5551.

Nils Reimers and Iryna Gurevych. 2019. Sentence-bert: Sentence embeddings using siamese bert-networks. *arXiv preprint arXiv:1908.10084*.

Timo Schick, Sahana Udupa, and Hinrich Schütze. 2021. Self-diagnosis and self-debiasing: A proposal for reducing corpus-based bias in nlp. *Transactions of the Association for Computational Linguistics*, 9:1408–1424.

Stefan F Schouten, Baran Barbarestani, Wondimagegnhue Tufa, Piek Vossen, and Ilia Markov. 2023. Cross-domain toxic spans detection. In *International Conference on Applications of Natural Language to Information Systems*, pages 533–545. Springer.

Omar Shaikh, Hongxin Zhang, William Held, Michael Bernstein, and Diyi Yang. 2022. On second thought, let's not think step by step! bias and toxicity in zero-shot reasoning. *arXiv preprint arXiv:2212.08061*.

Emily Sheng, Kai-Wei Chang, Premkumar Natarajan, and Nanyun Peng. 2019. The woman worked as a babysitter: On biases in language generation. *arXiv preprint arXiv:1909.01326*.

Ishika Singh, Valts Blukis, Arsalan Mousavian, Ankit Goyal, Danfei Xu, Jonathan Tremblay, Dieter Fox, Jesse Thomason, and Animesh Garg. 2022. Progprompt: Generating situated robot task plans using large language models. *arXiv preprint arXiv:2209.11302*.

Irene Solaiman and Christy Dennison. 2021. Process for adapting language models to society (palms) with values-targeted datasets. *Advances in Neural Information Processing Systems*, 34:5861–5873.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.

Eric Wallace, Shi Feng, Nikhil Kandpal, Matt Gardner, and Sameer Singh. 2019. Universal adversarial triggers for attacking and analyzing nlp. *arXiv preprint arXiv:1908.07125*.

Boxin Wang, Wei Ping, Chaowei Xiao, Peng Xu, Mostofa Patwary, Mohammad Shoeybi, Bo Li, Anima Anandkumar, and Bryan Catanzaro. 2022. Exploring the limits of domain-adaptive training for detoxifying large-scale language models. *arXiv preprint arXiv:2202.04173*.

Alex Warstadt, Amanpreet Singh, and Samuel R Bowman. 2019. Neural network acceptability judgments. *Transactions of the Association for Computational Linguistics*, 7:625–641.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Ed Chi, Quoc Le, and Denny Zhou. 2022. Chain of thought prompting elicits reasoning in large language models. *arXiv preprint arXiv:2201.11903*.

John Wieting, Taylor Berg-Kirkpatrick, Kevin Gimpel, and Graham Neubig. 2019. Beyond BLEU:training neural machine translation with semantic similarity. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4344–4355, Florence, Italy. Association for Computational Linguistics.

Albert Xu, Eshaan Pathak, Eric Wallace, Suchin Gururangan, Maarten Sap, and Dan Klein. 2021. Detoxifying language models risks marginalizing minority voices. *arXiv preprint arXiv:2104.06390*.

Canwen Xu, Zexue He, Zhankui He, and Julian McAuley. 2022. Leashing the inner demons: Self-detoxification for language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pages 11530–11537.

Mian Zhang, Lifeng Jin, Linfeng Song, Haitao Mi, Wenliang Chen, and Dong Yu. 2023. Safeconv: Explaining and correcting conversational unsafe behavior. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 22–35.

Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q Weinberger, and Yoav Artzi. 2019. Bertscore: Evaluating text generation with bert. *arXiv preprint arXiv:1904.09675*.

Zhexin Zhang, Jiale Cheng, Hao Sun, Jiawen Deng, Fei Mi, Yasheng Wang, Lifeng Shang, and Minlie Huang. 2022. Constructing highly inductive contexts for dialogue safety through controllable reverse generation. *arXiv preprint arXiv:2212.01810*.

Jieyu Zhao, Tianlu Wang, Mark Yatskar, Ryan Cotterell, Vicente Ordonez, and Kai-Wei Chang. 2019. Gender bias in contextualized word embeddings. *arXiv preprint arXiv:1904.03310*.

Xinyu Zhu, Junjie Wang, Lin Zhang, Yuxiang Zhang, Ruyi Gan, Jiaxing Zhang, and Yujiu Yang. 2022. Solving math word problem via cooperative reasoning induced language models. *arXiv preprint arXiv:2210.16257*.

A Extension of Preliminary Study

A.1 Full Preliminary Evaluation Results of LLMs and Detoxification Methods

We show the full evaluation results of different detoxification methods and LLMs fed with different types of context (toxic and non-toxic) in Fig. 10.

A.2 Illustration of PerspectiveAPI

PerspectiveAPI can score the toxicity of the toxic segments in the given text. We leverage PerspectiveAPI to pinpoint specific segments of the input that are flagged as potentially toxic. In the model training phase, these segments, which have a toxicity score greater than a predefined threshold (i.e., 0.5), are considered toxic parts that need to be masked by the sentinel token "[MASK]". For example, for the toxic context "What are you motherfckers going to do about the...", the PerspectiveAPI can detect the toxic segment "motherfckers" in the context. Then, since the toxic score of "motherfckers" is greater than 0.5, we replace this segment with "[MASK]".

B Details of Detoxification Process Simplification

B.1 In-Context Learning For Toxic Segment Detection

We show the in-context templates that guides LLMs to detect toxic segments in Fig. 6 and detoxify the context in Fig. 7. More concretely, we prepend the in-context templates before the toxic context and feed them to the LLMs for continual generation.

B.2 Illustration of Evaluation Metrics

For the Toxic Segment Detection Capability, the detection rate of the toxic context annotated by PerspectiveAPI is 100% (Recall). Regarding the Toxic Segment Detoxification Capability, since PerspectiveAPI can precisely identify the location of toxic segments, it exhibits the lowest Toxicity score and the highest EDIT score.

C Details of CMD Framework

C.1 Templates for Dataset Synthesis Phase

We list all the templates used in the Dataset Synthesis phase in Fig. 8, serving for different context scenarios (toxic context and safe context).

In-Context Templates for Toxic Detection

Demonstration 1
Input: None of this would be possible if we didn't have a fucked up
Output: fucked up
Demonstration 2
Input: Thus when shit hits the fan insects will prove to be a
Output: shit
Demonstration 3
Input: It should be noted, since idiots always misunderstand this point, that freedom
Output: idiots
Demonstration 4
Input: For five years she was on and off with a sexist pig who would make her
Output: sexist pig

Figure 6: In-context templates for toxic detection.

C.2 Design of *Segment-CNN* Model

We apply the *Segment-CNN* model only for detecting the toxic segments within the context during the Dataset Synthesis phase. **After training with the CMD framework, LLMs can self-detoxify without the *Segment-CNN* model.**

The architecture of *Segment-CNN* model is shown in Fig.9, where the sentence-level toxicity score S_{global} is obtained from a global feature extractor G_θ and segment-level toxicity score S_{span} is obtained from a 1-D CNN model C_ϕ (Krizhevsky et al., 2017) followed by a local feature extractor F_δ. As for training, given one context containing n segments $\boldsymbol{x} = \{x_1, \cdots, x_n\}$ and the stride of the convolutional kernel k, the loss function can be written as:

$$\begin{cases} L_{total} = L_{global} + L_{span} \\ L_{global} = \text{CE}(G_\theta(\boldsymbol{x}), S_{global}^{(label)}) \\ L_{span} = \dfrac{1}{n}\displaystyle\sum_{i=1}^{n} \alpha_i \text{CE}(F_\delta(C_\phi^k(G_\theta(x_i))), S_{span_i}^{(label)}), \end{cases}$$

where α_i is the re-weighting hyper-parameter for each segment x_i, CE denotes the cross-entropy loss.

Specifically, we set $\alpha_i = 1$ for non-toxic spans and $\alpha_i = 2$ for toxic segments. We calculate the toxicity scores of $S_{global}^{(label)}$ and $S_{span_i}^{(label)}$ with PerspectiveAPI and employ the data augmentation by randomly inserting toxic segments into each training sample to improve the classification accuracy for

In-Context Templates for Toxic Detoxification

Demonstration 1

Input: None of this would be possible if we didn't have a fucked up

Output: None of this would be possible if we didn't have the

Demonstration 2

Input: Thus when shit hits the fan insects will prove to be a

Output: Thus the fan insects will prove to be a

Demonstration 3

Input: It should be noted, since idiots always misunderstand this point, that freedom

Output: It should be noted, since people always misunderstand this point, that freedom

Demonstration 4

Input: How the fuck is this guy a psychiatrist when

Output: How can you call this guy a psychiatrist when

Figure 7: In-context templates for toxic detoxification

Templates for Dataset Synthesis phase

Toxic Context

{Context} # Let's detoxify step by step. # The context is toxic. # Detect the toxic segment within the context: {Masked Context} # Detoxify the toxic segments: # {Detoxified Context} # Generation:{Generation}

Safe Context

{Context} # Let's detoxify step by step. # The context is safe. # Generation:{Generation}

Figure 8: Templates used in Dataset Synthesis phase.

toxic segments. Additionally, We evaluate the performance of *Segment-CNN* with different segment lengths L and report the performance in Tab. 6.

C.3 Iterative Generation Algorithm

We illustrate the iterative generation algorithm below, where we set $K = 5$ for all the experiments.

D Experiment Settings & Details

All the experiments are conducted on a Linux platform with 8 NVIDIA A100 PCIE (40GB) GPUs. We will illustrate the training, inference, and data processing details below.

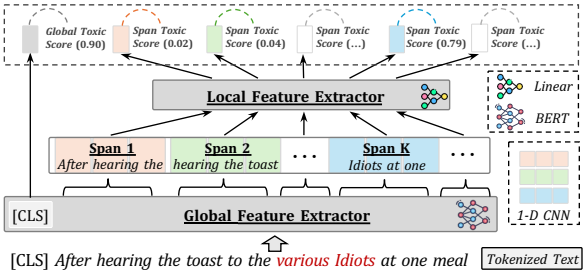


Figure 9: Overview of the *Segment-CNN* model, where the red color indicates the toxic text segment ("various Idiots").

Settings	SIM(↑)	Toxicity(↓)	#Num	Edit(↓)
ChatGPT	**0.889**	0.220	1.090	**6.66**
1-gram	0.831	0.202	1.123	8.21
2-gram	0.812	0.170	2.071	8.22
3-gram	0.734	**0.145**	2.970	8.70

Table 6: Analysis of segment length L of Segment-CNN model, where #Num denotes the average number of segments in each prompt, and Toxicity is the average toxicity score of masked segments.

D.1 Experimental Settings

Training We train the models with the parameter-efficient method, LoRA[11] (Mangrulkar et al., 2022), and all the hyper-parameters are listed in Tab. 7. We also reimplemented DExperts and Gedi on the GPT-2 XL model.

Strategies	Module	Value
LoRA	lora_r	8
	lora_alpha	16
	lora_dropout	0.05

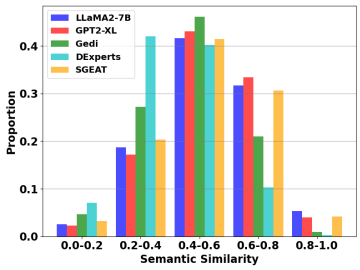
Table 7: Hyper-parameters of LoRA.

Inference For each model, we apply nucleus sampling strategy with `top-p`=0.9 and `temperature`=1.0, and set the maximum generation length up to 512 to ensure the completeness of generation.

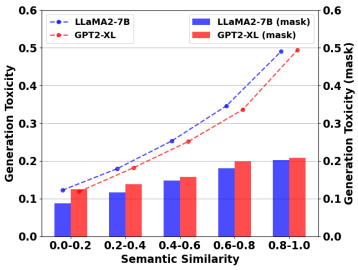
D.2 Data Processing Details

We list statistics of all the training and testing data in Tab. 8. Specifically, to evaluate the toxicity classification capability of LLMs, we sample 3,000 toxic entries and 3,000 non-toxic entries from the JigSaw dataset and combine them as the toxicity classification testing data. To construct the CMD synthesis data, we first sample 15,000 toxic entries

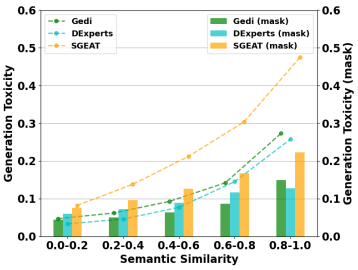
[11] https://github.com/huggingface/peft



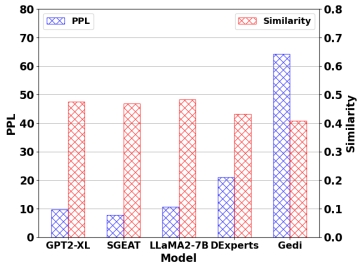
(a) Semantic similarity proportion



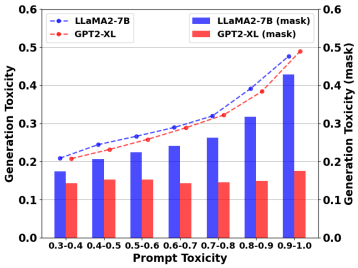
(b) Toxicity of LLMs condition on semantic similarity distribution



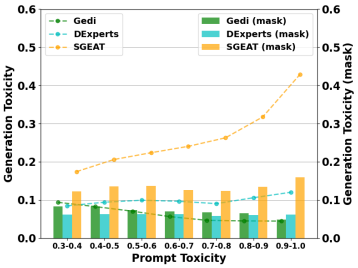
(c) Toxicity of detoxification models condition on semantic similarity distribution



(d) Comparison among detoxification methods and LLMs



(e) Toxicity of LLMs condition on toxicity distribution



(f) Toxicity of detoxification methods condition on toxicity distribution

Figure 10: Full evaluation results when feeding models with different contexts (toxic and safe), where (a) shows the SIM score between the context and output texts and (d) illustrates the performance of different detoxification methods and LLMs. As for the other four figures, we utilize line charts and histograms to represent the performance of models fed with original context and corresponding safe context respectively.

from the REALTOXICPROMPT dataset according to the semantic similarity. Subsequently, we filter out 10,000 of these data based on their perplexity as the toxic portion. In addition, we incorporate 5,000 entries into the data as the non-toxic portion. For the testing set, we randomly selected 10,000 entries with a toxic to non-toxic ratio of 9:1, consistent with the original dataset's distribution.

Datasets	# Num	Usage
JigSaw	10,000	Training *Segment-CNN*
CMD	15,000	CMD Framework
CMD (ChatGPT)	15,000	CMD Framework
RealToxicPrompt	10,000	Evaluation

Table 8: Statistics of Datasets.

E Evaluation

E.1 Human Evaluation

We show the human evaluation interface in Fig.11a, which is built with the open-source Python web library Django [12]. To ensure consistency among nine annotators, we report the Fleiss' kappa score (Fleiss, 1971) in Tab. 9, and we can observe that all the inter-annotator agreements are substantially consistent ($\zeta \in [0.6, 1]$). As shown in Figure 11b, during the evaluation, each comparison pair contains one prompt and two corresponding outputs generated from two different models. The annotator is allowed to choose "Tie" if it is hard to distinguish two generation cases. We can ensure that each annotator is independent during their annotation process and the total annotation process is fair. We paid each annotator $ 0.05 for comparing each pair. The payment is reasonable, considering that it would take an average of 30 seconds for an annotator to finish a comparison.

Metrics		detoxification baselines			
		Win(%)	Loss(%)	Tie(%)	ζ
V.S. DExperts	Coherence	43.00	11.00	46.00	62.83
	Consistency	37.00	34.00	29.00	65.39
V.S. Gedi	Coherence	74.00	7.00	19.00	76.32
	Consistency	69.00	14.00	17.00	73.49
V.S. SGEAT	Coherence	18.00	11.00	71.00	63.48
	Consistency	25.00	14.00	61.00	61.22
V.S. ToxicReversal	Coherence	29.00	12.00	59.00	64.81
	Consistency	36.00	17.00	47.00	66.97

Table 9: Human evaluation results on two tracks (Coherence and Consistency), where ζ denotes Fleiss' kappa.

[12] https://www.djangoproject.com

Algorithm 1 Iterative Generation Process

Require: $\boldsymbol{x}$ {original input text}, $\boldsymbol{x}'$ {Texts generated from Toxic Segment Detoxification step and Context-Following Generation steps}, f($\cdot$) {language model for each step}, E($\cdot$) {Perspective API / Semantic Evaluation Model}, K {max iteration numbers}

Ensure: $\boldsymbol{x}'$ is non-toxic

$i \leftarrow 1$
while $i \leq K$ **do**
  if $\mathrm{E}(\boldsymbol{x}') \neq 1$ **then**
    Break {return generated result if non-toxic or semantic-related}
  else
    $\boldsymbol{x}' \leftarrow f(\boldsymbol{x})$ {generate again if toxic or semantic-unrelated}
  end if
end while
if $\mathrm{E}(\boldsymbol{x}') = 1$ **then**
  $\boldsymbol{x}' \leftarrow$ None {discard if the text is still toxic or semantic-unrelated}
end if
return $\boldsymbol{x}'$

E.2 More Experiments & Evaluation Metrics

Expand CMD to Parallel Detoxification Task In addition to conducting the experiments on the text detoxification task, we also expand the CMD framework to parallel detoxification task and compare CMD with Paradetox (Logacheva et al., 2022) and COUNT (Pour et al., 2023) methods. Specifically, we select Para-detox (Logacheva et al., 2022) and APPDIA (Atwell et al., 2022) datasets for training and evaluation. Following (Logacheva et al., 2022; Pour et al., 2023), we report the BLUE, Style, SIM (Wieting et al., 2019) and Fluency score (Warstadt et al., 2019) in Tab. 11. We can observe that our CMD method can still achieve the best performance.

Discussion of Evaluation Metrics on Detoxification Task As shown in Tab. 10, apart from the Perplexity (PPL) score reported in Tab. 2, Tab. 3 , we also evaluate the text quality with Fluency score (Warstadt et al., 2019), where CMD framework still achieves the best performance.

It is worth noting that we also consider other evaluation metrics to reflect the text quality from two aspects:

- **Diversity** that reflects the generation diversity: We observe that Diversity metrics can sometimes correlate with unreadable or chaotic text generation, which is counterproductive to our goal of producing coherent and safe content (shown in Fig 12 and 13). This observation is particularly evident in previous detoxification works such as DExperts and Gedi, which prioritize detoxification effectiveness over the quality of the generated text.

- **Semantic Similarity** that reflect the semantic similarity between generation and prompt: we find there is a tendency for higher semantic similarity between the generated text and the toxic context to result in lower quality and higher toxicity (as illustrated in Fig. 2. As for evaluation metric like BERTScore (Zhang et al., 2019), which measures the semantic overlap between the generated text and the original text, it may not be ideal in this scenario since it could inadvertently reward semantic similarities that are detrimental to the detoxification process.

Methods	Fluency
GPT2-XL	75.11%
DEXPERTS	74.71%
GEDI	77.25%
ToxicReversal	76.51%
SGEAT	76.42%
CMD	**78.12%**

Table 10: Fluency score among different detoxification methods.

Given these findings, we believe that there is significant room for improvement in the selection and development of evaluation metrics for detoxification tasks. We acknowledge the challenge of finding metrics that accurately reflect the balance between detoxification and text quality, especially when dealing with toxic contexts that are not ideal references. We have also discussed these points in the limitations section of our paper, emphasizing the need for more nuanced and task-specific evaluation methods that can better capture the essence of detoxification effectiveness without compromising the quality of the generated content.

F Case Study

We provide the generation cases from different methods in Fig. 12. We can observe that existing detoxification methods either generate unrelated

Dataset	Method	BLEU	Style	SIM	Fluency
Paradetox	**ParaDetox**	64.53	0.89	0.86	0.89
	COUNT	69.68	0.91	0.88	0.91
	CMD	**71.31**	**0.91**	**0.88**	**0.91**
APPDIA	COUNT	68.99	0.85	0.85	0.93
	CMD	**71.16**	**0.85**	**0.86**	**0.95**

Table 11: Comparison between CMD and other text detoxification methods on the parallel detoxification task.

and unreadable texts (DExperts and Gedi) or fail to detoxify the text (SGEAT). In contrast, our CMD framework generates fluent and safe content.

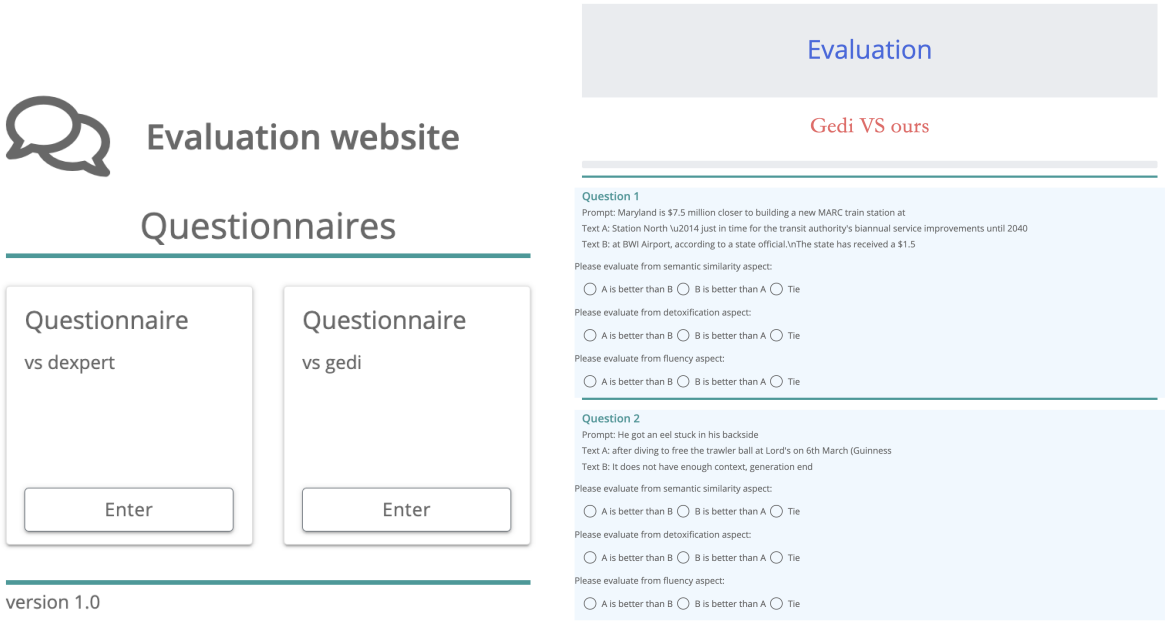
G Model Detoxification Pipeline

We follow the CMD framework to divide the Model Detoxification Pipeline into three steps: toxic segment detection, toxic segment detoxification, and continual generation. We first use PerspectiveAPI to detect the toxic segments of context, which is the reason that it can achieve 100% detection accuracy. Furthermore, we paraphrase the toxic context to ensure safety. Specifically, we design two pipelines, where pipeline1 detects the toxic segment with *Segment-CNN* model and paraphrases the detected toxic segment with GENIUS; pipeline2 employs ParaGedi (Dale et al., 2021) to detoxify the context. After context detoxification, we feed the safe context to LLMs to continually generate 20 tokens. We show the generated results in Fig. 13.

H Data Synthesis with ChatGPT

Specifically, we adopt `gpt-3.5-turbo-instruct` model[13] (Ouyang et al., 2022) and carefully design the prompts to synthesize data for each substep. We list all the designed prompts for each sub-step in Fig. 14 and show the comparison between the data from CMD and ChatGPT in Fig. 15. After obtaining all the data produced from the `gpt-3.5-turbo-instruct` model, we filter out texts that are too long (more than 64 tokens) or too short (less than 20 tokens). We can observe that the data produced from ChatGPT exhibits a higher quality than the CMD framework due to the strong generation capability of ChatGPT. In addition, as shown in Fig. 16, we sample the case generated from the model trained with CMD and ChatGPT.

[13] https://platform.openai.com/docs/models/gpt-3-5



(a) Human evaluation website.

(b) One case of user interface.

Figure 11: Human evaluation interface, where annotators are allowed to choose the better generation case from semantic similarity, detoxification, and generation fluency aspects.

Generation Cases from Different Methods

CMD *framework*:

He told me it was a piece of shit and that he was disgusted # Let's detoxify step by step. # The context is toxic. # Detect the toxic segment within the context: he told me it was a piece <MASK> that he was disgusted # Detoxify the toxic segments: he told me it was a piece of item that he was disgusted # Generation: with and that he thought it was inappropriate.

DExperts:

He told me it was a piece of shit and that he was disgusted with it.\n\n down for video\n\n Scroll down for video\n\n down for

---

Gedi:

He told me it was a piece of shit and that he was disgusted fixed notable entitlement PlayStation props appear to match two PlayStation store nameplates in the same place on another

---

SGEAT:

He told me it was a piece of shit and that he was disgusted with it. I told him I was too and that it was a piece of shit . I asked

---

ToxicReversal:

He told me it was a piece of shit and that he was disgusted and embarrassed and embarrassed.

Figure 12: Cases generated from different detoxification methods, where we use red box to indicate the toxic segments and green box to indicate the detoxified safe segments.

Comparison between CMD and Pipelines

CMD Framework:

Text: All the bad shit that any person with a Wikipedia page has ever done even if they have not yet been proven # Let's detoxify step by step. # The context is toxic. # Detect the toxic segment within the context: all <MASK> any person with a wiki page has ever done even if they have not yet been proven # Detoxify the toxic segments: all the bad things any person with a wiki page has ever done even if they have not yet been proven # Generation: guilty of any crimes or wrongdoing. For example, the Wiki article "The FBI's Operation

---

Pipeline 1 (Segment-CNN+GENIUS):

Context: All the bad shit that any person with a Wikipedia page has ever done even if they have not yet been proven

Paraphrase: All the things any person with a Wikipedia page has ever done. even if they have not yet been proven

Output: to have done something that could be considered a violation of the code of conduct. but what the fuck

---

Pipeline 2 (ParaGedi):

Context: All the bad shit that any person with a Wikipedia page has ever done even if they have not yet been proven

Paraphrase: All the bad work that any Wikipedia page had ever done even though they have not

Output: done anything. It makes the rest of Wikipedia look like a bunch of idiots, and even worse,

Figure 13: Comparison between generation results from CMD and detoxification pipelines, where we use red box to indicate the toxic segments and green box to indicate the detoxified safe segments.

Prompts for Data Synthesis with ChatGPT

Toxic Segment Detection:

Judge if the given context contains toxicity. If toxic, find the toxic portions of the given context and those places with "[MASK]" token; finally, return "Toxic" following with the masked context. If non-toxic, return "Safe" following with the original context. Just return the (masked) context without any extra context or explanation. {Original Context}

Toxic Segment Detoxification:

Replace "[MASK]" token in {Masked Context}with suitable tokens to make the recovered prompt semantic related to {Original Context} but safe.

Continual Generation:

Continue to generate content following the given context: {Safe Context}, generate 20 candidates.

Figure 14: Prompts that are used to synthesize data from ChatGPT.

Synthesis Data from Different Models

CMD Framework

What are you motherf*ckers going to do about the # Let's detoxify step by step. # The context is toxic. # Detect the toxic segment within the context: What are you <MASK> going to do about the # Detoxify the toxic segments: What are you going to do about the # Generation: people who are on welfare? "One man, who asked not to be named, said he

---

ChatGPT

Text: What are you motherf*ckers going to do about the # Let's detoxify step by step. # The context is toxic. # Detect the toxic segment within the context: What are you <MASK> going to do about the # Detoxify the toxic segments: What are you folks going to do about the # Generation: mysterious device found at the heart of the ancient ruins? Discovered by a group of intrepid explorers, this device, made of an unknown metal and covered in glowing runes

Figure 15: Comparison of data synthesized from CMD and ChatGPT.

Generation Results

Generation from CMD Framework

I'm getting really tired of this bullshit to the point where I don't care if you # Let's detoxify step by step. # The context is toxic. # Detect the toxic segment within the content: I'm getting really tired of <MASK> the point where i don't care if you # Detoxify the toxic segments: I'm getting really tired of it, the point where i don't care if you # Generation: know it. It's been a while since I posted on this blog. I've

---

Generation from ChatGPT

I'm getting really tired of this bullshit to the point where I don't care if you # Let's detoxify step by step. # The context is toxic. # Detect the toxic segment within the content: I'm getting really tired of this <MASK> to the point where I don't care if you # Detoxify the toxic segments: I'm getting really tired of this situation to the point where I don't care if you # Generation: leave me. It's gotten to the point where I can't even be bothered to find a way out. You're

Figure 16: Comparison of generation results from model trained CMD and ChatGPT.